

Python Programming Lab Manual

Functions, Recursion, Lambda Expressions, Scope,
Functional Programming & Decorators

Lab No.	Topic
1	Defining and Calling Functions
2	Types of Function Arguments
3	Recursion
4	Anonymous Functions (Lambda Expressions)
5	Global and Local Variables
6	Functional Programming: map(), filter(), reduce()
7	Decorators

Instructions: Write, execute and test each program in a Python IDE / Jupyter Notebook. Save each program with the file name suggested in the task (e.g. lab1_task1.py). Record the source code, sample input/output and your observations in your lab record. Attempt the Viva-Voce questions at the end of each lab.

LAB 1: Defining and Calling Functions

Objective: To understand how to define, call and document Python functions, and to use return values effectively.

Task 1: Simple Greeting Function

- Define a function `greet()` that takes a name as a parameter and prints "Hello, ! Welcome to Python."
- Call the function with at least 3 different names.
- Save the file as `lab1_task1.py`

Sample Output:

```
Hello, Asha! Welcome to Python.  
Hello, Ravi! Welcome to Python.
```

Task 2: Simple Interest Calculator

- Write a function `simple_interest(principal, rate, time)` that calculates and returns the simple interest using $SI = (P * R * T) / 100$.
- Call the function with values entered by the user and display the result.
- Add a docstring explaining what the function does.

Task 3: Even or Odd Checker

- Define a function `is_even(n)` that returns `True` if `n` is even and `False` otherwise.
- Write a driver program that reads 5 numbers from the user, calls `is_even()` for each, and prints whether each number is even or odd.

Task 4: Function Returning Multiple Values

- Write a function `stats(numbers)` that accepts a list of numbers and returns the minimum, maximum and average as a tuple.
- Unpack the returned tuple into three separate variables in the calling code and display them clearly.

Task 5: Temperature Converter (Nested Function Calls)

- Write a function `celsius_to_fahrenheit(c)` that converts Celsius to Fahrenheit.
- Write another function `fahrenheit_to_celsius(f)` that does the reverse conversion.
- Write a menu-driven program that repeatedly asks the user which conversion to perform, calling the appropriate function, until the user chooses to exit.

Viva-Voce Questions

- 1 What is the difference between a function definition and a function call?
- 2 What does a function return if it has no explicit return statement?
- 3 Can a Python function return more than one value? How?
- 4 What is the purpose of a docstring, and how do you access it programmatically?
- 5 What is the difference between a parameter and an argument?

LAB 2: Types of Function Arguments

Objective: To explore positional, keyword, default, variable-length (*args) and keyword variable-length (**kwargs) arguments in Python functions.

Task 1: Positional and Keyword Arguments

- Define a function `student_info(name, roll_no, branch)` that prints student details.
- Call it once using positional arguments and once using keyword arguments, in a different order, and verify that both calls give the same output.

Task 2: Default Arguments

- Write a function `calculate_price(price, tax_rate=18, discount=0)` that computes the final price after tax and discount, where `tax_rate` and `discount` have default values.
- Call the function: (a) with only the price, (b) with price and a custom `tax_rate`, and (c) with all three arguments overridden.

Task 3: Variable-Length Arguments (*args)

- Write a function `total_marks(*marks)` that accepts any number of subject marks and returns their sum and average.
- Test the function with 3 marks, 5 marks and 1 mark in separate calls.

Task 4: Keyword Variable-Length Arguments (**kwargs)

- Write a function `build_profile(**details)` that accepts an arbitrary number of keyword arguments (such as name, age, city, hobby) and prints them as a neatly formatted profile card.
- Call the function with different sets of keyword arguments to show its flexibility.

Task 5: Combining All Argument Types

- Write a function `order_summary(customer, *items, discount=0, **extra)` that:
 - Prints the customer name (positional)
 - Lists all ordered items (*args)
 - Applies a discount (default/keyword argument)
 - Prints any extra info such as `delivery_address` or `gift_wrap` (**kwargs)
- Call this function with a realistic combination of all four argument types in one call.

```
order_summary("Meera", "Laptop", "Mouse", discount=10,  
             delivery_address="Hyderabad", gift_wrap=True)
```

Viva-Voce Questions

- 1 What is the difference between *args and **kwargs?
- 2 Why must default arguments always appear after non-default arguments in a function definition?
- 3 What error occurs if you pass a positional argument after a keyword argument in a function call?
- 4 Can you mix *args and **kwargs in the same function signature? In what order must they appear?
- 5 How would you force certain arguments to be keyword-only?

LAB 3: Recursion

Objective: To understand recursive function design, base cases and recursive cases, and to compare recursion with iteration.

Task 1: Factorial Using Recursion

- Write a recursive function `factorial(n)` that computes $n!$ (n factorial).
- Add proper handling for $n = 0$ (base case) and negative input.
- Compare the recursive version with an iterative version and note the difference in code.

Task 2: Fibonacci Series Using Recursion

- Write a recursive function `fibonacci(n)` that returns the n -th term of the Fibonacci series.
- Print the first 15 terms of the series using this function.
- Observe and note how many times `fibonacci(5)` is recomputed while calculating `fibonacci(10)` (this motivates memoization).

Task 3: Sum of Digits and Digit Reversal

- Write a recursive function `sum_of_digits(n)` that returns the sum of the digits of a positive integer n .
- Write a second recursive function `reverse_number(n)` that returns n with its digits reversed.

Task 4: Power Function Using Recursion

- Write a recursive function `power(base, exp)` that computes base raised to exp without using the `**` operator.
- Extend it to handle $\text{exp} = 0$ and negative exponents.

Task 5: GCD Using Recursion (Euclidean Algorithm)

- Write a recursive function `gcd(a, b)` that computes the Greatest Common Divisor of two numbers using the Euclidean algorithm: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$, with $\text{gcd}(a, 0) = a$.
- Use this function to also compute the LCM of two numbers, using $\text{LCM}(a, b) = (a * b) / \text{GCD}(a, b)$.

Task 6: Tower of Hanoi

- Write a recursive function `tower_of_hanoi(n, source, auxiliary, destination)` that prints the sequence of moves required to solve the Tower of Hanoi puzzle for n disks.
- Run the program for $n = 3$ and $n = 4$, and count the total number of moves printed. Verify it matches $2^n - 1$.

Viva-Voce Questions

- 1 What is a base case, and why is it essential in every recursive function?
- 2 What happens if a recursive function has no base case, or the base case is unreachable?
- 3 What is the time complexity of the naive recursive Fibonacci function, and why?
- 4 What is a stack overflow / `RecursionError`, and how is it related to recursion depth in Python?
- 5 What is tail recursion? Does Python optimize tail-recursive calls?

LAB 4: Anonymous Functions: Lambda Expressions

Objective: To learn how to write and use single-expression anonymous functions using the `lambda` keyword.

Task 1: Basic Lambda Functions

- Write lambda expressions to: (a) square a number, (b) check if a number is positive, negative or zero was intended for `def`, so here just check if a number is even, (c) find the larger of two numbers.
- Assign each lambda to a variable and call it with sample inputs.

Task 2: Lambda with Conditional Expression

- Write a lambda function `grade = lambda marks: (...)` that returns 'Pass' if marks $\geq$ 40 else 'Fail', using a conditional expression inside the lambda.
- Test it with a list of 6 marks and print the result for each.

Task 3: Sorting with Lambda as key

- Given a list of tuples representing (student_name, marks), sort the list in descending order of marks using `sorted()` with a lambda as the key argument.
- Given a list of strings, sort them by their length using a lambda key.

```
students = [("Ravi", 78), ("Sita", 92), ("Amit", 65)]  
sorted(students, key=lambda s: s[1], reverse=True)
```

Task 4: Lambda Inside `map()` and `filter()`

- Given a list of numbers, use `map()` with a lambda to produce a list of their cubes.
- Given the same list, use `filter()` with a lambda to extract only numbers divisible by 3.

Task 5: Lambda for Dictionary Value Sorting

- Given a dictionary of item names mapped to prices, use `sorted()` with a lambda key to display the items from cheapest to most expensive.

Viva-Voce Questions

- 1 What is the key difference between a lambda function and a normal function defined with `def`?
- 2 Can a lambda function contain multiple statements or a return statement? Why or why not?
- 3 Why are lambda functions commonly used with `map()`, `filter()`, `sorted()` and `reduce()`?
- 4 Can you assign a lambda function to a variable and call it later? Is this considered good practice?
- 5 Can a lambda function take zero arguments, default arguments, or `*args/**kwargs`?

LAB 5: Global and Local Variables

Objective: To understand variable scope in Python, the global and nonlocal keywords, and common scope-related pitfalls.

Task 1: Local vs Global Scope

- Declare a global variable counter = 0 at the top of the program.
- Write a function show_local() that creates a local variable with the same name and prints it, then print the global variable outside the function.
- Explain in your record why the two values are different.

Task 2: Modifying a Global Variable Using the global Keyword

- Write a function increment_counter() that uses the global keyword to modify the global counter variable defined in Task 1, incrementing it by 1 each time it is called.
- Call the function 5 times and print the counter value after each call.

Task 3: UnboundLocalError Demonstration

- Write a function that tries to modify a global variable inside it without declaring it global, and observe/record the UnboundLocalError that Python raises.
- Fix the function using the global keyword and show that it now works correctly.

Task 4: Nested Functions and the nonlocal Keyword

- Write an outer function make_counter() that defines a local variable count = 0 and an inner function increment() that increases count using the nonlocal keyword.
- Have make_counter() return the inner function, then call it multiple times from outside and observe that the count persists between calls (this demonstrates a closure).

Task 5: Bank Account Simulation (Applied Scope Exercise)

- Write a program that maintains a global variable balance = 1000.
- Write functions deposit(amount) and withdraw(amount) that correctly update the global balance using the global keyword, with withdraw() also checking for insufficient funds.
- Write a menu-driven loop that lets the user deposit, withdraw, or check the balance until they choose to exit.

Viva-Voce Questions

- 1 What is the difference between local scope and global scope in Python?
- 2 Why is the global keyword required to modify a global variable inside a function, but not required to simply read it?
- 3 What is the LEGB rule in Python?
- 4 What is the purpose of the nonlocal keyword, and how does it differ from global?
- 5 What is a closure, and how does it relate to nonlocal variables?

LAB 6: Functional Programming: map(), filter(), reduce()

Objective: To apply functional programming techniques in Python using map(), filter(), and functools.reduce() to process sequences without explicit loops.

Task 1: Using map() to Transform Data

- Given a list of temperatures in Celsius, use map() with a named function (not a lambda) to convert them all to Fahrenheit.
- Given a list of strings, use map() to convert every string to uppercase.

Task 2: Using filter() to Select Data

- Given a list of integers from 1 to 50, use filter() to extract all prime numbers (write a helper function is_prime()).
- Given a list of words, use filter() to keep only palindromes.

Task 3: Using reduce() to Aggregate Data

- Import reduce from functools. Given a list of numbers, use reduce() to compute: (a) their product, (b) the maximum value, without using the built-in max() function.
- Use reduce() to concatenate a list of strings into a single sentence.

Task 4: Chaining map(), filter() and reduce()

- Given a list of numbers, write a single pipeline that: (1) filters out the odd numbers, (2) maps the remaining even numbers to their squares, and (3) reduces the squared values to their total sum.
- Solve the same problem again using a single list comprehension and sum(), and compare readability with your notes.

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = filter(lambda x: x % 2 == 0, nums)
squares = map(lambda x: x ** 2, evens)
total = reduce(lambda a, b: a + b, squares)
```

Task 5: Employee Records Processing

- Given a list of dictionaries, each representing an employee with keys 'name', 'department' and 'salary':
 - - Use filter() to select employees from a specific department.
 - - Use map() to give every selected employee a 10% salary hike (return new dictionaries; do not mutate the originals).
 - - Use reduce() to compute the total salary expenditure for that department after the hike.

Viva-Voce Questions

- 1 What do map(), filter() and reduce() have in common as functional programming tools?
- 2 Why must reduce() be imported from the functools module in Python 3, unlike map() and filter()?
- 3 What data type does map() and filter() return in Python 3? How would you convert the result into a list?
- 4 How could each of map(), filter() and reduce() be rewritten using list comprehensions or loops?
- 5 What is the initial value (initializer) argument of reduce(), and when is it necessary?

LAB 7: Decorators

Objective: To understand functions as first-class objects and to design and apply decorators for cleanly extending function behaviour.

Task 1: First-Class Function Warm-Up

- Demonstrate that functions are first-class objects in Python by: (a) assigning a function to a new variable and calling it through that variable, (b) passing a function as an argument to another function, (c) returning a function from another function.
- This task does not use @decorator syntax; it only prepares the concepts needed for it.

Task 2: A Basic Logging Decorator

- Write a decorator `log_call` that, when applied to any function, prints the function's name and its arguments before calling it, and prints the return value after the call.
- Apply `@log_call` to a simple `add(a, b)` function and test it.

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} args={args} kwargs={kwargs}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper
```

Task 3: Execution Time Decorator

- Write a decorator `timer` that measures and prints how long the decorated function takes to execute, using the `time` module.
- Apply it to a function that performs a computationally heavy task (e.g. summing numbers in a large range) and observe the timing output.

Task 4: Decorator with Arguments

- Write a parameterised decorator `repeat(n)` that, when applied to a function, calls the decorated function `n` times in a row each time it is invoked.
- Apply `@repeat(3)` to a function that prints a greeting, and verify it prints the greeting three times per call.

Task 5: Access-Control Decorator

- Write a decorator `require_login` that wraps a function and only allows it to execute if a global variable `is_logged_in` is `True`; otherwise it should print an access-denied message and not call the function.
- Demonstrate the decorator's behaviour both when `is_logged_in` is `True` and when it is `False`.
- Use `functools.wraps` in this decorator and explain in your record why it is good practice to use it.

Task 6: Stacking Multiple Decorators

- Apply both `@log_call` (from Task 2) and `@timer` (from Task 3) to the same function, stacked on top of each other.
- Run the program and note the order in which the decorators' effects appear in the output; explain why that order occurs.

Viva-Voce Questions

- 1 What is a decorator in Python, and what problem does it solve?

- 2 Why do decorator wrapper functions typically use `*args` and `**kwargs`?
- 3 What is `functools.wraps` used for, and what goes wrong if it is omitted?
- 4 How would you write a decorator that itself accepts arguments (like `@repeat(3)`)? What extra level of nesting does this require?
- 5 When multiple decorators are stacked on one function, in what order are they applied and in what order do they execute?

General Lab Guidelines

- Write the complete source code for every task in your lab record, along with sample input and output.
- Use meaningful variable and function names, and include comments/docstrings where appropriate.
- Test each program with at least one normal case and one edge case (e.g. empty list, zero, negative number).
- Save your programs using the naming convention labX_taskY.py, where X is the lab number and Y is the task number.
- Be prepared to answer the Viva-Voce questions at the end of each lab during evaluation.